

# Azure DevOps Services Permission Matrix: A Comprehensive Guide for IT Administrators

---

## Introduction

---

Azure DevOps Services provides a robust platform for managing the entire software development lifecycle. A critical aspect of administering Azure DevOps is the effective management of permissions, ensuring that users and services have appropriate access to resources while maintaining security and compliance. This report serves as a comprehensive guide for IT administrators, detailing the permission structure, roles, security groups, and best practices across Azure DevOps Services. Understanding and implementing these controls effectively is paramount for securing your organization's assets, streamlining workflows, and enabling teams to work efficiently.

This guide focuses on Azure DevOps Services (cloud) and covers permissions at the organization, project, repository, and pipeline levels. It aims to equip administrators with the knowledge to design, implement, and maintain a secure and scalable permission model.

## Overall Azure DevOps Security Model

---

Azure DevOps employs a hierarchical security model where permissions can be defined at multiple levels:

1. **Organization Level:** Permissions set at this level apply to all projects and resources within the organization.
2. **Project Level:** Permissions are scoped to a specific project, affecting its repositories, pipelines, work items, etc.
3. **Object Level:** Fine-grained permissions can be set on specific objects like Git branches, build pipelines, area paths, or service connections.

Permissions are typically managed by assigning users or groups to **security groups**, which have predefined or custom permission sets. Azure DevOps also uses **access levels** (e.g., Stakeholder, Basic, Visual Studio Subscriber) to control feature availability for users.

Key principles of the Azure DevOps security model include:

- \* **Inheritance:** Permissions are generally inherited from parent levels (e.g., project permissions inherit from organization settings, object permissions inherit from project settings) unless explicitly overridden.
- \* **Explicit Permissions:** Permissions explicitly set to **Allow** or **Deny** override inherited permissions.
- \* **Deny Precedence:** A **Deny** permission typically overrides an **Allow** permission. This is a crucial aspect for restricting access.
- \* **Group-Based Management:** The recommended practice is to manage permissions through groups rather than assigning them to individual users, simplifying administration and auditing.

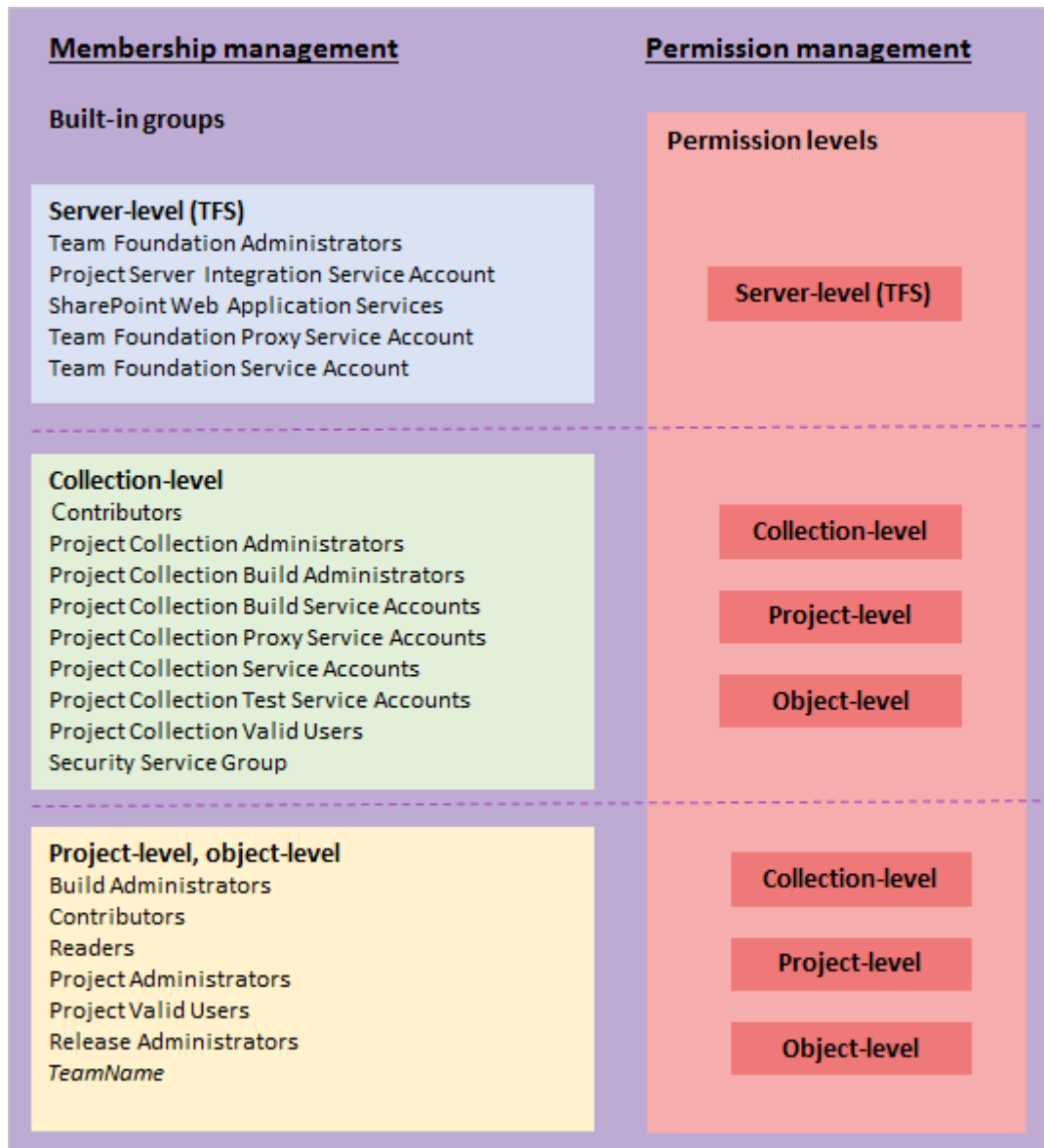


Figure 1: Overview of Azure DevOps Permission Management (Source: Microsoft Learn)

## Organization-Level Permissions and Security Groups

Organization-level settings and permissions govern the entire Azure DevOps instance for your company. Managing these effectively is the foundation of a secure DevOps environment.

### Overview of Organization-Level Security

All security groups in Azure DevOps are technically organization-level entities, even if their scope of influence is limited to specific projects. They are managed centrally within the organization settings.

## Default Organization-Level Security Groups

Azure DevOps creates several default security groups at the organization (or collection) level. The most powerful of these is:

- **Project Collection Administrators (PCA):**
  - **Capabilities and Responsibilities:** Members of this group have the highest level of permissions across the entire organization/collection. They can:
    - Manage all organization settings, policies, and billing.
    - Create, edit, and delete projects.
    - Manage users, group memberships, and access levels for all projects.
    - Install, uninstall, and manage extensions for the organization.
    - Administer all resources, including agent pools, service connections, and deployment groups at the organization level.
    - Modify permissions for any user or group at any level.
    - Access and manage audit logs.
  - The organization owner is automatically a member of this group.
  - It's critical to limit membership in this group to only essential personnel.

Other notable collection-level groups include:

- \* **Project Collection Build Administrators:** Manage build resources and permissions across the collection.
- \* **Project Collection Valid Users:** Includes all users and groups with access to the collection. Membership is managed automatically.
- \* **Project-Scoped Users:** A special group that, when enabled via the "Limit user visibility and collaboration to specific projects" preview feature, restricts users' visibility to only those projects they are explicitly added to.

## Managing Organization-Level Permissions

Permissions at the organization level are managed through **Organization Settings > Security > Permissions**.

- **Assigning Permissions:** While some organization-wide settings are controlled by PCA membership, specific service permissions (like for agent pools or extensions) can be managed for other groups.
- **Role-Based Access Control (RBAC):** Leverage Azure Active Directory (Azure AD) / Microsoft Entra ID groups to manage membership in Azure DevOps groups. This allows for centralized identity management and can automate access level assignments using **Group Rules**.
- **Group Rules:** Available in Azure DevOps Services, group rules allow administrators to define rules that automatically assign access levels (e.g., Basic, Stakeholder) and project membership to users based on their Azure AD group memberships.

## Best Practices for Organization-Level Security

- **Limit PCA Membership:** Strictly control who is a member of the Project Collection Administrators group. Have at least two members for redundancy but keep the number minimal.
- **Use Azure AD/Entra ID Integration:** For scalable user management, integrate Azure DevOps with Azure AD/Entra ID. Assign Azure AD groups to Azure DevOps groups.
- **Do Not Modify Default Group Permissions:** Avoid altering the default permissions of built-in groups like Project Collection Administrators. If custom permission sets are needed, create new custom groups.
- **Regular Audits:** Periodically review memberships of high-privilege groups and organization-level permission settings. Utilize Azure DevOps audit logs.
- **Descriptive Naming Conventions:** Use clear and consistent naming conventions for custom groups (e.g., `[CompanyName] - [Role]` or `[CompanyName] ServiceAccount [ServiceName]`).

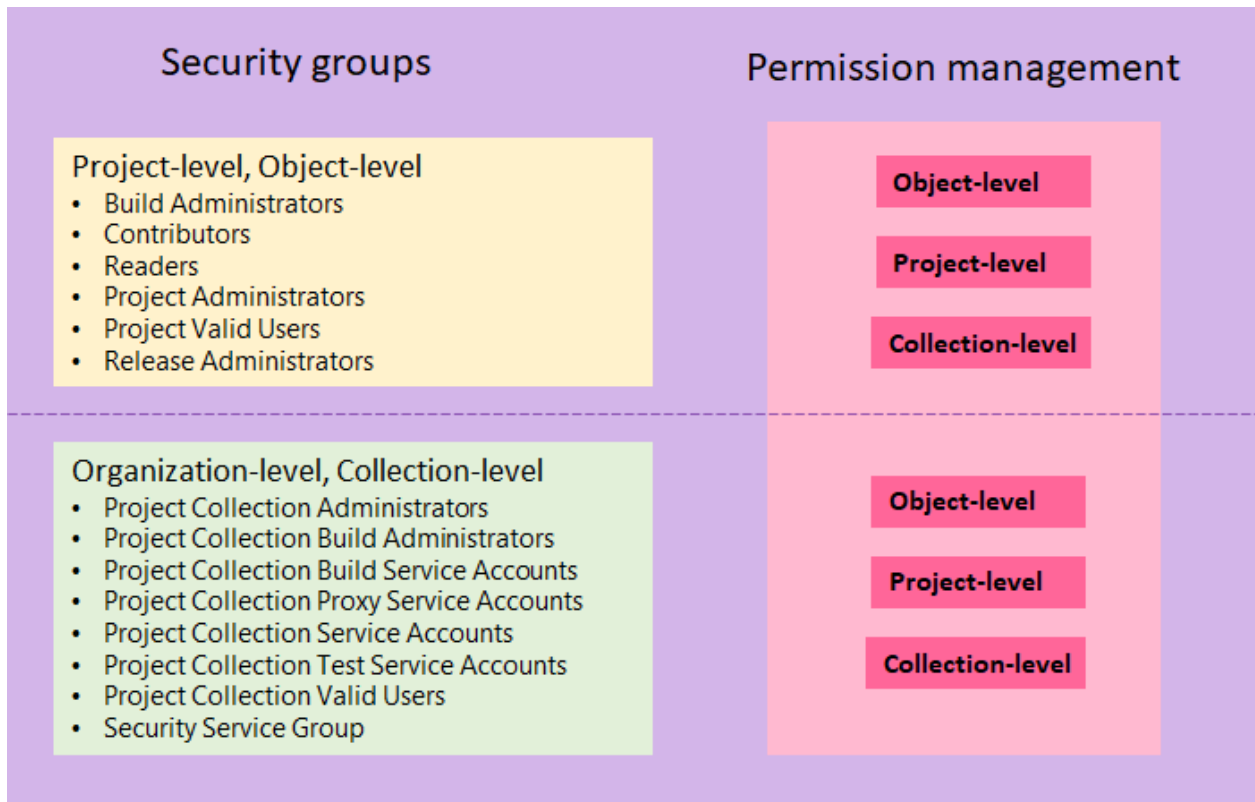


Figure 2: Security Groups and Permission Management in Azure DevOps (Source: Microsoft Learn)

## Project-Level Permissions and Roles

Permissions at the project level control access to resources and functionalities within a specific Azure DevOps project.

### Overview of Project-Level Security

Each project in Azure DevOps has its own set of security groups and permissions, which are initially inherited or set by default upon project creation. Project Administrators can customize these permissions to suit the project's needs.

### Default Project-Level Security Groups and Roles

Azure DevOps creates several default security groups when a new project is created:

- **Project Administrators:**
  - **Capabilities:** Full control over the project. They can manage project settings, permissions, teams, area and iteration paths, service connections scoped to the project, and agent pools/queues for the project. They can add members to the project and manage project-level group memberships.
- **Contributors:**
  - **Capabilities:** This group is intended for users who actively contribute to the project. Members can typically:
    - Create, edit, and delete work items.
    - Add, modify, and delete code in repositories (Git and TFVC).
    - Create and manage build and release pipelines.
    - Create and manage test plans and test cases.
    - Add tags to work items and code.
    - Configure team settings, backlogs, and boards.
  - By default, the team group created with the project is a member of the Contributors group.

- **Readers:**

- **Capabilities:** This group is for users who need view-only access to project information. Members can typically:
  - View work items, backlogs, and boards.
  - View code in repositories.
  - View build and release pipeline definitions and results.
  - View dashboards and project reports.
- They cannot make changes to any project artifacts.

- **Build Administrators:**

- **Capabilities:** Manage build resources and permissions for the project, including build definitions (pipelines) and agent queues.

- **Release Administrators:**

- **Capabilities:** Manage release pipelines, deployment groups, and related settings for the project. This group is typically created when the first release pipeline is defined.

- **Project Valid Users:**

- **Capabilities:** Includes all users and groups that have been added to the project. This group is primarily used to grant general access to view project information. Its membership is managed automatically.

## Managing Project-Level Permissions

Project-level permissions are managed via **Project Settings > Security > Permissions**.

- **Adding Users/Groups:** Administrators can add users or Azure AD/DevOps groups to project-level security groups.
- **Setting Permissions:** For each group or user, specific permissions for various services (Boards, Repos, Pipelines, Test Plans, Artifacts) can be set to **Allow**, **Deny**, or **Not set** (inherit).
- **Team Permissions:** Each team within a project also has an associated security group. Team administrators can manage team settings, but project-level permissions still apply.

## Best Practices for Project-Level Security

- **Principle of Least Privilege:** Grant users and groups only the permissions necessary to perform their roles.
- **Group-Based Assignments:** Assign permissions to Azure DevOps groups or Azure AD groups rather than individual users.
- **Utilize Built-in Groups:** Leverage the default groups (Contributors, Readers, etc.) as much as possible for standard roles. Create custom groups for more specific permission sets.
- **Avoid Overuse of Project Administrator:** Limit the number of Project Administrators. Delegate specific administrative tasks using custom groups with tailored permissions if needed.
- **Regular Review:** Periodically review project-level permissions and group memberships.

## Repository Permissions and Branch Policies

Securing source code is paramount. Azure DevOps provides granular permissions for repositories and powerful branch policies to enforce development standards.

### Overview of Repository Security

Within Azure Repos (Git and TFVC), permissions can be set at the repository level and, for Git, at the individual branch level.

## Repository-Level Permissions (Git)

Common permissions for Git repositories include:

- **Read:** Allows users to clone, fetch, and view the repository content and history. (Typically granted to Project Valid Users).
- **Contribute:** Allows users to push new commits to existing branches, create new branches, and create pull requests. (Typically granted to Contributors).
- **Force push (rewrite history, delete branches and tags):** Allows users to forcibly overwrite history on a branch or delete branches/tags. This is a powerful permission and should be highly restricted.
- **Manage permissions:** Allows users to change security settings for the repository or its branches.
- **Create branch:** Allows users to create new branches in the repository.
- **Create tag:** Allows users to create tags (typically for releases).
- **Bypass policies when pushing:** Allows users to push changes directly to a branch even if branch policies are not met.
- **Bypass policies when completing pull requests:** Allows users to complete pull requests that fail branch policy checks.

Permissions can be managed from **Project Settings > Repositories > [Select Repository] > Security**, or for all repositories. For individual branches, navigate to **Repos > Branches > [Select Branch] > ... (More options) > Branch security**.

## Branch Policies

Branch policies are essential for protecting important branches (e.g., `main`, `develop`, `release/*`) and ensuring code quality and collaboration standards. They are configured for specific branches or branch patterns.

### Common Branch Policies:

- **Require a minimum number of reviewers:** Enforces that pull requests (PRs) are reviewed and approved by a specified number of team members before merging.
  - Options include allowing/disallowing requestors to approve their own changes, prohibiting the most recent pusher from approving, and resetting votes on new pushes.
- **Check for linked work items:** Requires PRs to be linked to one or more work items, ensuring traceability.
- **Check for comment resolution:** Ensures all comments on a PR are addressed (resolved or marked as “won’t fix”) before merging.
- **Limit merge types:** Restricts the merge strategies allowed (e.g., allow squash merge, disallow rebase).
- **Build validation:** Requires a specific build pipeline to run successfully on the PR changes before merging. This ensures code compiles and passes automated tests.
- **Status checks:** Requires other services (e.g., external CI/CD systems, security scanners) to post a successful status to the PR.
- **Automatically include reviewers:** Adds specific users or groups as reviewers automatically when a PR targets the branch.

## Configuring Repository Permissions and Branch Policies

### To set repository permissions:

1. Navigate to **Project Settings > Repositories**.
2. Select the specific repository or “All Repositories”.
3. Go to the **Security** tab.
4. Search for and select a user or group.
5. Set the desired permissions (Allow/Deny/Not set).

**To configure branch policies:**

1. Navigate to **Repos > Branches**.
2. Find the branch you want to protect, select the ... (**More options**) icon, and choose **Branch policies**.
3. Enable and configure the desired policies from the available options (e.g., minimum reviewers, build validation).

\* For example, to enforce two reviewers: Toggle “Require a minimum number of reviewers” to On, set “Required number of reviewers” to 2.

\* To enforce a build: Toggle “Build validation” to On, click “+”, select the build pipeline, and configure triggers and display name.

## Best Practices for Repository and Branch Security

- **Protect Key Branches:** Apply stringent branch policies to `main`, `develop`, and release branches.
- **Restrict Direct Pushes:** For protected branches, disallow direct pushes by setting the `Contribute` permission appropriately and enforce changes through PRs with policies.
- **Use Force push Sparingly:** Grant `Force push` permission only to a very limited set of administrators or lead developers, and only for specific scenarios (e.g., cleaning up private feature branches).
- **Enforce Code Reviews:** Always require at least one or two reviewers for PRs into important branches.
- **Automate Validation:** Use build validation and status checks to automate code quality and security checks.
- **Least Privilege for Permissions:** Grant only necessary repository permissions. For example, most developers only need `Contribute` and `Create branch`.
- **Regularly Review Policies:** Ensure branch policies are still relevant and effective as team practices evolve.

## Pipeline Permissions and Security

Azure Pipelines enables CI/CD automation. Securing pipelines is crucial to protect your build and deployment processes, as well as the environments they target.

### Overview of Pipeline Security

Pipeline security involves controlling who can define, manage, and run pipelines, as well as securing the resources pipelines use, such as service connections, agent pools, and variable groups.

### Pipeline-Level Permissions

Permissions for build and release pipelines can be managed at the project level (for all pipelines) or at the object level (for individual pipelines).

#### Common Pipeline Permissions:

- **View builds/View build pipeline (View releases/View release pipeline):** Allows users to see pipeline definitions and run history.
- **Edit build pipeline (Edit release pipeline):** Allows users to modify pipeline definitions.
- **Delete build pipeline (Delete release pipeline):** Allows users to remove pipeline definitions.
- **Queue builds (Create releases):** Allows users to manually trigger pipeline runs.
- **Manage build qualities (Manage deployment/release tags):** Allows users to manage tags or quality labels on pipeline runs.
- **Administer build permissions (Administer release permissions):** Allows users to manage security settings for pipelines.
- **Stop builds (Manage deployments):** Allows users to cancel running pipelines or manage deployments.

### Managing Pipeline Permissions:

- \* **Project-Level: Project Settings > Pipelines (under Pipelines section) > ... (More options) > Manage security.**
- \* **Object-Level (Individual Pipeline):** Open the specific pipeline, click ... **(More options) > Manage security.**

Inheritance can be disabled at the object level to set specific permissions for a pipeline.

## Securing Service Connections

Service connections allow pipelines to connect to external services (e.g., Azure subscriptions, Kubernetes clusters, artifact repositories).

- \* **Scope Permissions Tightly:** When creating a service connection (e.g., for Azure Resource Manager), scope it to the narrowest possible level (e.g., a specific resource group or management group, not the entire subscription) and grant it only the necessary roles.
- \* **Use Workload Identity Federation:** For Azure service connections, prefer workload identity federation over service principals with secrets for improved security and easier credential management.
- \* **Restrict Usage:** In the service connection's security settings, control which pipelines or users can use it. Add pipeline permissions and user permissions.
- \* **Approvals and Checks:** Configure approvals and checks on service connections (and environments) to enforce manual validation or automated checks before a pipeline stage using the connection can proceed.

## Managing Secrets in Pipelines

- **Variable Groups:** Store secrets and sensitive variables in Azure DevOps Variable Groups. Link Azure Key Vault to a variable group to securely fetch secrets at runtime. Mark variables as "secret" (lock icon) to mask them in logs.
- **Azure Key Vault Task:** Use the Azure Key Vault task in pipelines to download secrets directly from Key Vault.
- **Avoid Hardcoding Secrets:** Never store secrets directly in pipeline YAML files or scripts.
- **Limit Secret Exposure:** Ensure that scripts or tasks do not inadvertently print secrets to logs.

## Agent Pool Security

- **Microsoft-Hosted Agents:** These are managed by Microsoft and provide a fresh VM for each job. They are generally secure for public projects and many private project scenarios.
- **Self-Hosted Agents:** If you use self-hosted agents:
  - Keep the agent software and underlying OS patched and up-to-date.
  - Run agents in isolated network environments if they access sensitive resources.
  - Grant the agent service account minimal necessary permissions on the host machine.
  - Control who can administer agent pools and queues. Permissions are managed at Organization Settings > Agent Pools or Project Settings > Agent Pools.
  - Assign the **User** role to groups that need to use the pool in their pipelines. Restrict the **Administrator** role.

## Best Practices for Pipeline Security

- **RBAC for Pipelines:** Use security groups to manage pipeline permissions.
- **Least Privilege:** Grant users and service connections only the permissions they absolutely need.
- **Secure YAML Pipelines:**
  - Protect the `azure-pipelines.yml` file in your repository using branch policies. Changes to this file directly impact the pipeline's execution.
  - Use templates for reusable and secure pipeline logic. Templates can be stored in a central, secured repository.
  - Limit the scope of tokens available to jobs (e.g., `Build.AccessToken` ).



- **Protect Environments:** For deployment pipelines, use Environments in Azure DevOps. Secure environments with approvals, checks (e.g., branch control, business hours), and user permissions.
- **Audit Regularly:** Review pipeline permissions, service connection usage, and audit logs for suspicious activity.
- **Branch Control for Triggers:** Restrict pipeline triggers to specific branches (e.g., only run deployment pipelines from release branches).

## Default Roles and Their Capabilities Consolidated

---

Azure DevOps uses a system of default security groups to assign common sets of permissions. Understanding these roles is key to effective permission management.

| Role/Group                               | Level        | Primary Capabilities                                                                                                                                | Typical Access Level |
|------------------------------------------|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| <b>Project Collection Administrators</b> | Organization | Full control over the organization/collection: manage projects, users, billing, extensions, organization-wide settings, and all permissions.        | Visual Studio/Basic  |
| <b>Project Administrators</b>            | Project      | Full control over a specific project: manage project settings, teams, permissions, area/iteration paths, project-scoped service connections.        | Visual Studio/Basic  |
| <b>Contributors</b>                      | Project      | Actively contribute to project: manage work items, code (commit, push, branch), build/release pipelines, test plans.                                | Basic                |
| <b>Readers</b>                           | Project      | View-only access to project artifacts: view work items, code, pipeline results, dashboards, reports. Cannot make changes.                           | Stakeholder/Basic    |
| <b>Build Administrators</b>              | Project      | Manage build resources for the project: define/edit build pipelines, manage agent queues for the project.                                           | Basic                |
| <b>Release Administrators</b>            | Project      | Manage release resources for the project: define/edit release pipelines, manage deployment groups, project-scoped service connections for releases. | Basic                |
| <b>Team Administrator</b>                | Team         | Manages team assets and settings: configure                                                                                                         | Basic                |

| Role/Group          | Level      | Primary Capabilities                                                                                                                           | Typical Access Level |
|---------------------|------------|------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
|                     |            | team backlogs, boards, sprints, dashboards, and team membership. Not a security group but a role.                                              |                      |
| <b>Stakeholders</b> | User-based | Limited access, primarily for work item tracking, viewing dashboards, and providing feedback. Cannot access code or build/release definitions. | Stakeholder          |

**Note:** Access Levels (Stakeholder, Basic, Basic + Test Plans, Visual Studio Subscriber) determine which features a user can access, while permissions (Allow/Deny) on specific actions are granted via security group membership.

## Best Practices for Azure DevOps Permission Management

Implementing a robust permission strategy involves adhering to established best practices:

### 1. Principle of Least Privilege (PoLP):

- Grant users, groups, and service accounts only the minimum permissions required to perform their tasks.
- Avoid granting broad administrative rights unless absolutely necessary.

### 2. Role-Based Access Control (RBAC):

- Define roles within your organization (e.g., Developer, QA Engineer, Release Manager, Project Manager).
- Create custom Azure DevOps security groups or map Azure AD groups to these roles.
- Assign permissions to these groups rather than individual users.

### 3. Use Azure AD / Microsoft Entra ID Groups:

- Integrate Azure DevOps with Azure AD/Entra ID for centralized identity management.
- Manage group memberships in Azure AD, and assign these Azure AD groups to Azure DevOps security groups. This simplifies onboarding/offboarding and allows for dynamic group membership.
- Utilize Azure DevOps Group Rules to automate access level assignment and project membership based on Azure AD group membership.

### 4. Manage Default Groups Carefully:

- Understand the default permissions of built-in groups (e.g., Contributors, Readers, Project Administrators).
- **Avoid modifying the permissions of default Azure DevOps groups.** If custom permissions are needed, create new groups. This prevents confusion and unexpected behavior if Microsoft updates default group permissions.

### 5. Auditing and Regular Reviews:

- Periodically review user access, group memberships, and assigned permissions.
- Utilize Azure DevOps audit logs (Organization Settings > Auditing) to track permission changes, group modifications, and other security-relevant events.
- Remove inactive users and revoke unnecessary permissions promptly.

## 6. Minimize Use of “Deny” Permissions:

- While “Deny” overrides “Allow”, extensive use of “Deny” can make permission troubleshooting complex. Prefer a model where users are granted specific “Allow” permissions and lack permissions for actions they shouldn’t perform. Use “Deny” strategically for specific lockdown scenarios.

## 7. Automation of Permission Management:

- For large organizations, consider automating permission assignments using the Azure DevOps REST API or CLI.
- Tools like Terraform or Bicep can manage some Azure DevOps configurations, including permissions, as code.

## 8. Clear Naming Conventions:

- Establish and use clear, consistent naming conventions for custom security groups (e.g., `[ProjectName]-[Role]-[PermissionScope]`, `ADO_Contributors_ProjectX`). This improves clarity and manageability.

## 9. Secure Service Accounts and PATs:

- For service accounts or automated processes, prefer service principals with workload identity federation or managed identities over Personal Access Tokens (PATs).
- If PATs must be used, scope them narrowly with minimal necessary permissions and set short expiration dates. Regularly review and revoke unused PATs.

# Common Permission Scenarios for Scaling Organizations

As organizations grow, managing Azure DevOps permissions at scale presents unique challenges and requires strategic planning.

## Structuring for Large Enterprises

- **Single Organization Strategy:** Generally, it’s recommended to use a **single Azure DevOps organization** for the entire enterprise. This facilitates centralized governance, easier cross-project collaboration, unified billing, and consistent policy application. Multiple projects can then be created within this organization to isolate different business units, product lines, or large initiatives.
- **Multiple Projects:** Use projects to segregate work, manage distinct teams, and apply different process templates or security boundaries. Azure DevOps supports up to 1,000 projects per organization.
- **Teams within Projects:** Utilize teams within projects to further organize work. Each team gets its own backlog, boards, and dashboards. Teams also have associated security groups, which can be used for fine-grained permission management within a project.

## Managing Cross-Team and Cross-Project Access

- **Shared Security Groups:** Create organization-level Azure AD groups or Azure DevOps groups for roles that span multiple projects (e.g., “Enterprise Architects,” “Security Reviewers”). Add these groups to relevant projects with appropriate permissions.
- **Area Paths and Iteration Paths:** Use area paths to control visibility and access to work items for different teams within the same project. Permissions can be set on area paths.
- **Service Connections and Agent Pools:** Share service connections and agent pools at the organization level if they are used by multiple projects, but ensure their usage is properly secured and restricted to authorized pipelines.

## Delegated Administration

- While Project Collection Administrators have ultimate control, delegate project-level administration by adding trusted users to the **Project Administrators** group for specific projects.
- For even more granular delegation, create custom groups with specific administrative permissions (e.g., a group that can only manage area paths for a project).

## Antipatterns to Avoid at Scale

- **Direct User Permission Assignments:** This becomes unmanageable and impossible to audit effectively in large organizations. Always use groups.
- **Modifying Default Azure DevOps Groups:** As mentioned, this can lead to confusion and break expected behavior. Create custom groups instead.
- **Overuse of Administrator Groups:** Granting too many users Project Administrator or, worse, Project Collection Administrator rights significantly increases risk.
- **Using Teams Solely for Permissions:** While teams have security groups, their primary purpose is work organization. If you only need a permission container without team tools (boards, backlogs), use a plain security group.
- **Inconsistent Naming Conventions:** Lack of clear naming for groups and projects makes it hard to understand and manage permissions.
- **Relying on “Deny” Extensively:** Complicates troubleshooting permission issues.

## Leveraging Group Rules and Azure AD

- **Group Rules:** Automate the assignment of access levels (e.g., Basic, Stakeholder) and project membership based on users' Azure AD group memberships. This is crucial for scaling user management.
- **Dynamic Azure AD Groups:** Use dynamic Azure AD groups (based on user attributes) to further automate membership in roles that are then mapped to Azure DevOps permissions.

## Security Considerations and Compliance Aspects

---

Beyond direct permission settings, a holistic approach to security and compliance is essential for any Azure DevOps environment.

### Aligning with Industry Standards

- Azure DevOps Services complies with various industry standards like ISO/IEC 27001, SOC 1/2/3, and GDPR.
- Your organization is responsible for configuring Azure DevOps and implementing processes to meet your specific compliance requirements based on these standards. This includes:
  - Enforcing strong authentication (MFA).
  - Implementing data protection policies.
  - Ensuring proper access controls (as detailed in this report).
  - Maintaining audit trails.

### Implementing Zero Trust Principles

Adopt a Zero Trust approach (“never trust, always verify”) for your DevOps environment:

- \* **Verify Explicitly:** Authenticate and authorize based on all available data points, including user identity, location, device health, service or workload, data classification, and anomalies.
- \* **Use Least Privileged Access:** Grant just-in-time and just-enough-access (JIT/JEA), risk-based adaptive policies, and data protection.
- \* **Assume Breach:** Minimize blast radius for breaches and prevent lateral movement by segmenting access. Verify end-to-end encryption and use analytics to get visibility, drive threat detection, and improve defenses.

## Data Protection and Network Security

- **Data Encryption:** Azure DevOps encrypts data at rest and in transit. Ensure your integrations and custom solutions also follow encryption best practices.
- **Network Controls:**
  - **IP Allowlisting:** If applicable, use Azure AD Conditional Access policies to restrict access to Azure DevOps from trusted IP address ranges.
  - **Private Endpoints/Service Endpoints:** For enhanced security, consider using Azure Private Link for Azure DevOps if your organization requires traffic to stay on the Microsoft backbone network.
  - **Self-Hosted Agent Security:** Secure the network where self-hosted agents run, especially if they access on-premises resources.

## Auditing for Compliance

- **Azure DevOps Audit Logs:** Regularly review audit logs (Organization Settings > Auditing) to monitor user activity, permission changes, policy modifications, and other significant events.
- **Audit Streams:** Stream audit logs to external SIEM (Security Information and Event Management) systems like Azure Sentinel for advanced analysis, alerting, and long-term retention.
- **Compliance Policies in Pipelines:** Enforce compliance checks within your CI/CD pipelines (e.g., security scans, license compliance checks, policy-as-code).

## Managing External Guests

- **Control Invitations:** If your organization policy allows, you can restrict invitations to external guests or allow them only from specific domains.
- **Azure AD B2B:** Manage external guest users through Azure AD B2B collaboration. Assign them to specific Azure AD groups that are then used to grant permissions in Azure DevOps.
- **Regular Review:** Periodically review external guest access and remove users who no longer require it.

## Removing Unnecessary Users and Service Accounts

- **Inactive Users:** Regularly identify and remove or disable accounts of users who have left the organization or no longer need access.
- **Service Account Hygiene:**
  - Use dedicated service principals or managed identities for automation instead of user accounts or PATs where possible.
  - Regularly review permissions granted to service accounts/principals.
  - Revoke PATs that are no longer needed or have overly broad scopes.

## Conclusion

Effectively managing permissions in Azure DevOps Services is a cornerstone of a secure, compliant, and efficient software development environment. By understanding the hierarchical security model, leveraging security groups, adhering to the principle of least privilege, and implementing best practices for organization, project, repository, and pipeline security, IT administrators can create a robust framework that supports their organization's needs.

For scaling organizations, a strategic approach involving Azure AD/Entra ID integration, automation through group rules, consistent naming conventions, and regular auditing is crucial. By treating permission management as an ongoing process of review and refinement, administrators can ensure that their Azure DevOps environment remains secure and aligned with business objectives, empowering development teams while safeguarding valuable assets. This guide provides the foundational knowledge to build and maintain such a system.

## References

---

- [About permissions and security groups - Azure DevOps](https://learn.microsoft.com/en-us/azure/devops/organizations/security/about-permissions?view=azure-devops) (https://learn.microsoft.com/en-us/azure/devops/organizations/security/about-permissions?view=azure-devops)
- [About projects and scaling your organization - Azure DevOps](https://learn.microsoft.com/en-us/azure/devops/organizations/projects/about-projects?view=azure-devops) (https://learn.microsoft.com/en-us/azure/devops/organizations/projects/about-projects?view=azure-devops)
- [All the Places to Set Permissions in Azure DevOps](https://marcoussy.com/2022/05/16/all-the-places-to-set-permissions-in-azure-devops/) (https://marcoussy.com/2022/05/16/all-the-places-to-set-permissions-in-azure-devops/)
- [Arnica Blog: Managing Azure DevOps Access Levels & Permissions](https://www.arnica.io/blog/managing-granular-permissions-in-azure-devops) (https://www.arnica.io/blog/managing-granular-permissions-in-azure-devops)
- [Azure Boards Explained: How to Structure Projects, Permissions, and Work Tracking at Scale](https://www.harjtech.com/blogs/azure-boards-explained-how-to-structure-projects-permissions-and-work-tracking-at-scale) (https://www.harjtech.com/blogs/azure-boards-explained-how-to-structure-projects-permissions-and-work-tracking-at-scale)
- [Azure DevOps CLI documentation](https://docs.microsoft.com/en-us/cli/azure/repos/policy?view=azure-cli-latest) (https://docs.microsoft.com/en-us/cli/azure/repos/policy?view=azure-cli-latest)
- [Azure DevOps Permissions Guide](https://www.azuredevopsguide.com/permissions-of-azure-devops-organization-owner/) (https://www.azuredevopsguide.com/permissions-of-azure-devops-organization-owner/)
- [Azure DevOps Permissions - Quick Reference](https://learn.microsoft.com/en-us/azure/devops/organizations/security/permissions/permissions?view=azure-devops) (https://learn.microsoft.com/en-us/azure/devops/organizations/security/permissions/permissions?view=azure-devops)
- [Azure DevOps Permissions/Security best practices - Reddit](https://www.reddit.com/r/azuredevops/comments/meecd-j7/azure_devops_permissionssecurity_best_practices/) (https://www.reddit.com/r/azuredevops/comments/meecd-j7/azure\_devops\_permissionssecurity\_best\_practices/)
- [Azure DevOps Repository Permissions](https://learn.microsoft.com/en-us/azure/devops/repos/git/branch-permissions?view=azure-devops) (https://learn.microsoft.com/en-us/azure/devops/repos/git/branch-permissions?view=azure-devops)
- [Azure DevOps Service Connection Security - Engineering Playbook](https://microsoft.github.io/code-with-engineering-playbook/CI-CD/dev-sec-ops/azure-devops-service-connection-security/) (https://microsoft.github.io/code-with-engineering-playbook/CI-CD/dev-sec-ops/azure-devops-service-connection-security/)
- [Azure DevOps Blog: Setting up Repository Permissions](https://devblogs.microsoft.com/premier-developer/azure-devops-setting-up-repository-permissions/) (https://devblogs.microsoft.com/premier-developer/azure-devops-setting-up-repository-permissions/)
- [Branch policies and settings - Azure DevOps](https://learn.microsoft.com/en-us/azure/devops/repos/git/branch-policies?view=azure-devops) (https://learn.microsoft.com/en-us/azure/devops/repos/git/branch-policies?view=azure-devops)
- [C# Corner: Working with Branch Policies in Azure DevOps](https://www.c-sharpcorner.com/article/working-with-branch-policies-in-azure-devops/) (https://www.c-sharpcorner.com/article/working-with-branch-policies-in-azure-devops/)
- [Default permissions quick reference - Azure DevOps](https://learn.microsoft.com/en-us/azure/devops/organizations/security/permissions/permissions?view=azure-devops) (https://learn.microsoft.com/en-us/azure/devops/organizations/security/permissions/permissions?view=azure-devops)
- [Design and implement permissions and security groups in Azure DevOps](https://notes.kodekloud.com/docs/AZ-400/Design-and-Implement-Authentication-and-Authorization-Methods/Design-and-implement-permissions-and-security-groups-in-Azure-DevOps) (https://notes.kodekloud.com/docs/AZ-400/Design-and-Implement-Authentication-and-Authorization-Methods/Design-and-implement-permissions-and-security-groups-in-Azure-DevOps)
- [DEV Community: Branch Policies in Azure Repos](https://dev.to/evdbogaard/branch-policies-in-azure-repos-11c5) (https://dev.to/evdbogaard/branch-policies-in-azure-repos-11c5)
- [DevOps.dev: Enterprise-Grade Azure DevOps — Security, CI/CD & Best Practices](https://devops.dev/enterprise-grade-azure-devops-security-ci-cd-best-practices-09fd739e7976) (https://devops.dev/enterprise-grade-azure-devops-security-ci-cd-best-practices-09fd739e7976)
- [Enterprise-level Azure DevOps permissions from the trenches](https://veegens.wordpress.com/2021/07/19/enterprise-level-azure-devops-permissions-from-the-trenches/) (https://veegens.wordpress.com/2021/07/19/enterprise-level-azure-devops-permissions-from-the-trenches/)
- [Get started with permissions, access levels, and security groups - Azure DevOps](https://learn.microsoft.com/en-us/azure/devops/organizations/security/about-permissions?view=azure-devops) (https://learn.microsoft.com/en-us/azure/devops/organizations/security/about-permissions?view=azure-devops)
- [KTL Solutions: Azure DevOps Security and Compliance Best Practices](https://ktlsolutions.com/azure-devops-security-and-compliance-best-practices/) (https://ktlsolutions.com/azure-devops-security-and-compliance-best-practices/)
- [Make your Azure DevOps secure - Microsoft Learn](https://learn.microsoft.com/en-us/azure/devops/organizations/security/security-overview?view=azure-devops) (https://learn.microsoft.com/en-us/azure/devops/organizations/security/security-overview?view=azure-devops)
- [Manage security in Azure Pipelines - Microsoft Learn](https://learn.microsoft.com/en-us/azure/devops/pipelines/policies/permissions?view=azure-devops) (https://learn.microsoft.com/en-us/azure/devops/pipelines/policies/permissions?view=azure-devops)
- [Medium: Securing Azure DevOps: Best Practices and Examples](https://medium.com/@malshikahiruni1999/securing-azure-devops-best-practices-and-examples-c101f84fdd2a) (https://medium.com/@malshikahiruni1999/securing-azure-devops-best-practices-and-examples-c101f84fdd2a)

[Microsoft Azure DevOps Documentation](https://learn.microsoft.com/en-us/azure/devops/organizations/security/permissions?view=azure-devops) (https://learn.microsoft.com/en-us/azure/devops/organizations/security/permissions?view=azure-devops)

[Microsoft Learn: Git branch policies and settings](https://learn.microsoft.com/en-us/azure/devops/repos/git/branch-policies?view=azure-devops) (https://learn.microsoft.com/en-us/azure/devops/repos/git/branch-policies?view=azure-devops)

[Microsoft Permissions and Role Lookup Guide](https://learn.microsoft.com/en-us/azure/devops/organizations/security/permissions-lookup-guide?view=azure-devops) (https://learn.microsoft.com/en-us/azure/devops/organizations/security/permissions-lookup-guide?view=azure-devops)

[Microsoft REST API](https://learn.microsoft.com/en-us/rest/api/azure/devops/security/permissions?view=azure-devops) (https://learn.microsoft.com/en-us/rest/api/azure/devops/security/permissions?view=azure-devops)

[Nvisia: Secure Azure DevOps Practices for Enhanced Compliance](https://www.nvisia.com/insights/secure-and-compliant-azure-devops) (https://www.nvisia.com/insights/secure-and-compliant-azure-devops)

[Permissions, security groups, and service accounts reference - Azure DevOps](https://learn.microsoft.com/en-us/azure/devops/organizations/security/permissions?view=azure-devops) (https://learn.microsoft.com/en-us/azure/devops/organizations/security/permissions?view=azure-devops)

[Set branch permissions - Azure DevOps](https://learn.microsoft.com/en-us/azure/devops/repos/git/set-branch-permissions?view=azure-devops) (https://learn.microsoft.com/en-us/azure/devops/repos/git/set-branch-permissions?view=azure-devops)

[Stack Overflow: Permissions for Project Collection Administrators](https://stackoverflow.com/questions/78414465/azure-devops-permissions-for-project-collections-administrators-effectiveallow) (https://stackoverflow.com/questions/78414465/azure-devops-permissions-for-project-collections-administrators-effectiveallow)

[StatusNeo: Best Practices for Securing Azure DevOps Pipelines](https://statusneo.com/best-practices-for-securing-azure-devops-pipelines/) (https://statusneo.com/best-practices-for-securing-azure-devops-pipelines/)

[Team Foundation Server Default Groups, Permissions, and Roles](https://devblogs.microsoft.com/devops/team-foundation-server-default-groups-permissions-and-roles-2/) (https://devblogs.microsoft.com/devops/team-foundation-server-default-groups-permissions-and-roles-2/)

[Xebia: Enterprise-Level Azure DevOps Permissions From The Trenches](https://xebia.com/blog/enterprise-level-azure-devops-permissions-from-the-trenches/) (https://xebia.com/blog/enterprise-level-azure-devops-permissions-from-the-trenches/)